

Report - 25.02.2026

25.02.2026

First rule check : some older uploads from earlier in this chat session are expired,
→ but for what you asked ****right now**** were good I have `main.tex`, all ****final clean CSVs****, and the core PDFs (DESTINO + your consolidated literature notes). If
→ you later want me to use a specific expired PDF again, youll need to re-upload
→ that one.

I opened `main.tex` and read it line-by-line, and I cross-checked every number in your
→ tables against the ****final clean datasets**** you produced today (`\*\_clean.csv`).
→ Your tables are ****actually correct**** (good news). Below is the deep evaluation you
→ asked for: roadmap alignment, DESTINO alignment, data quality, loose ends, and
→ improvement plan.

1) How well did we operate vs the roadmap?

Your professors roadmap (second bible) is:

1. literature review
2. define comparison criteria incl feasibility
3. select OSS projects
4. compute & compare multiple metrics (compute done; compare deferred correctly)
5. select one metric for PoC (correctly not done yet)
6. PoC smart contract (correctly not done yet)
7. discussion/writing (Phase I report is solid and now consistent)

****Roadmap compliance score: 9.5/10****

Why not 10? Only because we still have one unresolved extraction limitation (timestamp
→ semantics) that you already documented, and we should either fix it or plan
→ sensitivity analysis.

2) Is the report aligned with DESTINO?

DESTINO (primary bible) is mainly about evidence: ****hash + link + signature****, then
→ store ****metrics + tags + storage persistence****, and keep things auditable.

What is aligned

Your report reflects DESTINO-friendly thinking in multiple places:

- * You explicitly state ****no PoC metric is selected yet**** and selection happens after
→ Phase II (good: matches professor constraint and prevents premature bias).
- * You consistently frame feasibility as hybrid is recommended (off-chain computation +
→ on-chain compact commitments), which is the only realistic way to implement this
→ within constraints.

* You include ****storage persistence**** and ****software-type tags**** in the dataset freeze
↳ (DESTINO metadata requirement).
* You keep the pipeline ****auditable****: frozen repos/tags/windows, cleaning logs, and
↳ canonical datasets.

What is not yet implemented (but thats OK at Phase I)

DESTINOs on-chain workflow isnt implemented yet (signature verification, storing
↳ hashes on ledger, emitting events). But Phase I is supposed to end before PoC. So
↳ this isnt a failure its just ****future work****.

****DESTINO alignment at Phase I score: 8/10****

If you want 910/10 later, youll add one section in Month 4 describing how your PoC
↳ stores:

* ``archive_hash``, ``archive_link``, ``signer_id``, ``tags``, ``storage_persistence``, plus
↳ ``metric_value`` and ``evidence_hash``.

3) Deep data audit vs the final CSVs

I validated your final clean bundle:

Dataset shape (matches exactly what we planned)

* 4 repos frozen
* 8 stable tags per repo ****32 tags total**** (``versions.csv``)
* 7 windows per repo ****28 windows total**** (``windows.csv``)
* LOC per tag ****32 rows**** (``loc_per_tag.csv``)
* clean developer-window dataset ****273 rows**** (``metrics_window_dev_clean.csv``)
* clean window totals ****28 rows**** (``metrics_window_totals_clean.csv``)
* clean ownership dev shares ****273 rows**** (``ownership_dev_shares_clean.csv``)
* clean ownership summaries ****28 rows**** (``ownership_summary_clean.csv``)

Your report tables are correct

I re-computed and matched the numbers you printed, including:

* min/max/mean window durations per repo (e.g., requests min window 0.26 days)
* total commits and churn per repo
* global totals: ****902 commits****, ****58,298 churn****
* LOC min/max/mean per repo
* M4 concentration means and gini values

So the tables arent correct fear is understandable (because in past versions they
↳ were), but ****in this current `main.tex` the numbers match the final clean CSVs****.

****Data correctness score: 9/10****

The remaining 1 point is exactly one known limitation:

Loose end (still true): timestamp semantics (author-date drift)

You correctly document that some ``first_commit`` values can be earlier than the window
↳ boundary because timestamps are author-date, not committer-date. This doesnt break
↳ churn or commit counts, but it can distort cadence in some analyses.

You already wrote it as a known limitation, which is good.

4) What are the loose ends we must fix or attend to?

These are the real ones (not nice to have).

Loose end A Decide how youll handle cadence timestamps

In your report you plan sensitivity checks on timestamp semantics great.

What you must do later (Phase II):

- * either re-extract cadence using committer-date (`%cI`) OR
- * keep author-date but treat cadence primarily as commit counts per window + per-day
↪ normalized rates and mention timestamp limitation.

Loose end B Normalize by window duration in Phase II (you already noted this)

Windows vary wildly:

- * requests has a 0.26-day window (!!)
- * click has windows up to ~490 days

So Phase II must use:

- * commits/day
- * churn/day
- * optionally churn/(KLOCûday)

You already mention this in Section `\ref{sec:evaluation_unit}` and in the window
↪ duration table; thats exactly correct. The loose end is: ****make sure you actually**
↪ do it in Month 3**.

Loose end C M5 maintenance responsiveness

Right now M5 is listed as design extension thats fine.
But to avoid confusion, tweak wording slightly:

- * In shortlist table you say: not implemented in baseline PoC
- * Better: not computed in Phase I; documented as PoC extension
Because PoC isnt started yet.

Loose end D References file / compilation

Your ``main.tex`` ends with:

```
```\latex
\nocite{*}
\bibliographystyle{plain}
\bibliography{references}
```\pre>
```

If your ``references.bib`` is missing or only contains 1 entry, the PDF will show 1
↪ reference.

So the loose end is not your LaTeX its the BibTeX file and compilation steps.
Make sure the project includes ``references.bib`` with entries for:

```
* destino
* blockchainsurvey
* smartmetrics2023
* scvulncomplexity
  and that you compile: `pdflatex bibtex pdflatex pdflatex`.
```

5) Where can the report improve (and why)?

Your report is already a strong **Phase I technical report**. Improvements now are
→ mainly about clarity and thesis polish.

Improvement 1 Reduce duplicated phrasing, keep structure tighter

Right now you explain Phase I freeze/pipeline/cleaning in multiple sections (which is
→ okay), but you can tighten by:

```
* Keeping one Phase I summary paragraph at the end of each chapter
* And referencing the execution ledger table (`\ref{tab:phase1_execution_ledger}`)
  → instead of repeating steps
```

This improves readability without losing content.

Improvement 2 Add a Data Dictionary section (high value)

This is a classic thesis improvement: define exactly what each CSV column means.

For example:

```
* `metrics_window_dev_clean.csv` columns: commits, added, deleted, churn, active_days,
  → is_bot, etc.
* `ownership_summary_clean.csv`: top1/top3 share definitions, gini meaning
```

This makes the thesis readable for non-experts and is a common examiner expectation.

Improvement 3 Make DESTINO mapping explicit (one short subsection)

Add a small subsection like:

```
* How Phase I datasets map to DESTINO ledger records
* Mention: these CSV outputs are what later become metric records committed on chain
  → (hash + metadata + value).
```

That will raise DESTINO alignment score.

Improvement 4 Replace N/A in rubric template table

In `Table~\ref{tab:rubric\_template}`, you put N/A in every cell.
That can confuse a reviewer (they might think not applicable rather than not scored
→ yet).

Better: leave cells blank or put (deferred) once in caption.

6) Scorecard for what weve done so far

Heres a fair grading for Phase I:

```
* **Roadmap discipline:** 9.5/10
* **DESTINO alignment (phase-appropriate):** 8/10
* **Dataset freeze quality:** 10/10
* **Metric extraction correctness (M1M4):** 9/10
* **Cleaning/validation rigor:** 9.5/10
* **Report quality as Phase I technical report:** 8.5/10
  (mainly limited by bibliography wiring + missing data dictionary + small wording
  ↪ tweaks)
```

Overall: **9/10 for Phase I execution**.

Next thing to fix (before you pause)

If you want the fastest quality jump without starting Month 3:

1. add a **Data Dictionary** section
2. ensure `references.bib` exists + contains all citations + compiles
3. tweak M5 wording (not computed in Phase I; design extension)

If you want, paste your `references.bib` (or upload it), and Ill verify the
↪ bibliography keys match the ``\cite{...}`` keys in your `main.tex` so you dont get
↪ empty references.

1 Solutions in Details

First rule check : **some older uploads have expired in this chat session.** For the
↳ loose ends were addressing now, were still fine because we have your **final clean**
↳ CSVs, ``main.tex``, ``DESTINO4MasterThesis.pdf``, and your consolidated literature
↳ notes. If you want me to use/cite any specific expired paper again, just re-upload
↳ it.

Now I'll zoom in on each loose end exactly as you asked:

1. why its a loose end
2. how it affects the project
3. top 3 best solutions
4. what to do (detailed steps)

Loose end 1 Timestamp semantics (author date drift)

1) Why its a loose end

In your extracted dataset, some time fields (``first_commit``, ``last_commit``, and
↳ therefore ``active_days``) are based on Git **author date**. In real Git history,
↳ author date can be much older than when the commit actually entered the branch
↳ (e.g., merges, cherry-picks, rebases). Thats why you observed cases where
↳ ``first_commit`` appears **before** the windows ``from_date``.

This is a loose end because Month 3 includes **cadence/stability over time**, and time
↳ semantics must be consistent to avoid misleading results.

2) How it affects our project

* **Does NOT affect:** M2 churn, commit counts, window membership (the commit range
↳ ``from_hash..to_hash`` is correct).
* **Can affect:**

- * ``active_days`` (if author date is old, active days may fall outside the window)
- * any time gap and cadence interpretation using ``first_commit/last_commit``
- * any per-day normalization if you accidentally use timestamps rather than window
↳ duration

If we ignore it, the professor could fairly say:

> Your cadence metric is not well-defined; youre mixing timestamps that dont
↳ correspond to window boundaries.

3) Top 3 best solutions (with detailed steps)

Solution A (Best, simplest, most defensible): Re-extract cadence timestamps using
↳ **committer date**

Why it solves it: committer date reflects when the commit was actually applied to
↳ the branch history. Its the right choice for activity in this window.

What to do (exactly):

1. Modify the extraction command in your script to output **committer date** instead
↳ of author date.

```

    * Replace `%ad` with `%cI` (ISO 8601 committer date).
2. Recompute for each window:

    * `active_days` from committer date
    * `first_commit` / `last_commit` from committer date
3. Keep everything else the same (added/deleted/churn remains unchanged).
4. Re-run:

    * `extract_window_metrics.py` produce `metrics_window_dev_committer.csv`
    * then run the same cleaning script `metrics_window_dev_clean_committer.csv`
5. In the thesis, state:

    * Cadence features are based on committer timestamps; churn is based on diff stats;
    ↪ windows are tag-to-tag.

**Pros:** most accurate, clean story, no ambiguity.
**Cons:** you'll regenerate 23 CSVs (but it's fast).

---

### Solution B (Good, minimal disruption): Keep author-date but redefine cadence to be
↪ **count-based only**

**Why it solves it:** you avoid relying on questionable timestamps; cadence becomes
↪ number of commits per window and commit rate using window duration, not
↪ timestamp-based first/last.

**What to do:**

1. In Month 3, define cadence as:

    * `commits_per_window`
    * `commits_per_day = commits / window_duration_days` (window duration comes from
    ↪ tag dates, not commit timestamps)
2. Drop or downplay:

    * `first_commit`, `last_commit`, `active_days`
3. In your thesis:

    * explicitly say author-date drift exists; hence cadence is operationalized using
    ↪ counts and window length, not raw commit timestamps.

**Pros:** no re-extraction.
**Cons:** you lose active days nuance; cadence becomes simpler.

---

### Solution C (Strongest academically, slightly more work): Dual-run robustness check
↪ (author vs committer)

**Why it solves it:** you can show the professor you tested sensitivity. If results
↪ are similar, you gain credibility.

**What to do:**

1. Keep the current dataset as author-date cadence.
2. Create a second dataset committer-date cadence (Solution A extraction).
3. Run Month 3 cadence analyses twice (or at least compare summary stats):

```

```

    * stability metrics with author-date and committer-date
4. Report in thesis:

    * Results are robust (or not) to timestamp semantics.

**Pros:** best scientific practice; strong defense.
**Cons:** more work and more reporting.

---

# Loose end 2 Window duration variability (very short vs very long windows)

## 1) Why its a loose end

Because your windows are defined by release tags, the time between releases is not
↳ constant. Some windows are tiny (hours/days), others are months. That makes raw
↳ counts misleading.

Example intuition:

* 50 commits in 2 days 50 commits in 200 days (very different intensity)

This is a loose end because Month 3 includes cross-window comparisons and stability
↳ across versions.

## 2) How it affects our project

* Without normalization, metrics like churn/commits will look unstable simply because
↳ one window is longer.
* Cross-project comparisons become unfair because projects have different release
↳ rhythms.
* It also affects ownership concentration: long windows accumulate more contributors.

## 3) Top 3 best solutions

### Solution A (Best default): Rate-normalize by window duration (days)

**Why it solves it:** makes windows comparable.

**What to do:**

1. Compute `duration_days = (to_date - from_date)`.
2. Add derived metrics in Month 3:

    * `commits_per_day`
    * `churn_per_day`
    * optionally `active_days_per_day = active_days / duration_days` (if you fix
    ↳ timestamps)
3. Use rate metrics for stability comparisons (RQ1), while still reporting raw totals
↳ as context.

**Pros:** simple, standard, defensible.
**Cons:** very short windows can inflate rates (need guard).

---

### Solution B (Also strong): Use fixed time windows (monthly) instead of tag windows
↳ (for stability)

```


****Why it solves it:**** makes all windows equal length.

****What to do:****

1. Keep tag windows for release-to-release story.
2. Add a parallel monthly-window dataset (12 months) for cadence stability.
3. Compare stability using monthly windows; use tag windows as secondary.

****Pros:**** very comparable and stable.

****Cons:**** adds dataset complexity; you already froze tag windows.

Solution C (Minimal): Keep tag windows, but apply ****minimum window length rule****

****Why it solves it:**** prevents nonsense windows (like <1 day) from distorting results.

****What to do:****

1. If ``duration_days < X`` (e.g. 7 days):

- * merge it with the next window, or

- * treat as an outlier window and exclude from stability stats (documented)

2. Justify: extremely short release gaps produce noisy windows.

****Pros:**** simple, keeps tag framing.

****Cons:**** changes frozen windows; must be documented carefully.

Loose end 3 Ownership results bots and identity consistency

1) Why its a loose end

You correctly flagged bots (``is_bot``). But if you dont explicitly decide how bots are
↳ treated in results, you risk confusion:

- * Bots can contribute churn/commits (dependency bumps, formatting) which distort
↳ ownership and concentration.

Identity consistency (email aliases) was mostly fixed via cleaning, but across windows
↳ a person could still have multiple emails (future issue in Round 2).

2) How it affects the project

- * Ownership concentration (top1/top3, gini) can be inflated if bots produce lots of
↳ small commits.

- * Professor may ask: Are you measuring developers or automation?

3) Top 3 best solutions

Solution A (Best practice): Report both with bots and without bots

****Why it solves it:**** shows robustness and transparency.

****What to do:****

1. In Month 3, compute ownership stats twice:

```

    * including bots
    * excluding bots (`is_bot=False`)
2. In thesis: show main results (no-bot) and a sensitivity note (with-bot).

**Pros:** strongest, defensible.
**Cons:** slightly more reporting.

---

### Solution B (Minimal): Exclude bots from all ownership calculations (keep in raw
↪ data)

**Why it solves it:** keeps ownership about humans.

**What to do:**

1. Filter rows where `is_bot=True` before calculating:

    * shares
    * top1/top3
    * gini
2. Still keep bot rows in raw dataset for auditability.

**Pros:** clean ownership story.
**Cons:** you must justify why bots are excluded.

---

### Solution C (Advanced): Keep bots, but down-weight them

**Why it solves it:** recognizes bots contribute but dont represent developer quality.

**What to do:**

1. Assign `bot_weight = 0.1` and multiply bot commits/churn before computing shares.
2. Explain as a heuristic; include sensitivity results.

**Pros:** nuanced.
**Cons:** heuristic may be challenged; I dont recommend for minimum thesis.

---

# Loose end 4 Bibliography and compilation (why only 1 reference shows up)

## 1) Why its a loose end

Your `main.tex` can be correct but if:

* `references.bib` is missing, or
* BibTeX isnt run, or
* there are no \cite calls,
  the references section will appear empty or with 1 item.

## 2) How it affects the project

* A thesis with missing references looks unfinished and can hurt evaluation
  ↪ immediately.
* It breaks credibility: claims arent anchored.

```

3) Top 3 best solutions

Solution A (Best): Use BibTeX correctly and add real citations

****What to do:****

1. Ensure `references.bib` contains at least these 4 entries:

- * DESTINO proposal
- * blockchain survey
- * PLOS smart-metrics
- * smart-contract complexity/vulnerability paper

2. Add 610 citations in the text.

3. Compile in this order:

- * pdflatex
- * bibtex
- * pdflatex
- * pdflatex

Solution B (Debug mode): add `\nocite{\*}` temporarily

****What to do:****

1. Add `\nocite{\*}` to force printing all bib entries.

2. Once stable, remove it and keep only cited refs.

Solution C (If BibTeX is painful): switch to `biblatex` + biber

****What to do:****

1. Use `biblatex` package and `\addbibresource{references.bib}`

2. Compile with `biber` instead of `bibtex`

My recommendation (minimum work, maximum thesis strength)

If you want the cleanest path with minimum risk:

1. ****Fix cadence timestamps with committer date**** (Solution 1A)
2. ****Use rate-normalized metrics**** for stability (Solution 2A)
3. ****Report ownership both with and without bots**** (Solution 3A)
4. ****Fix bibliography**** with BibTeX (Solution 4A)

If you say go, I'll give you ****exact Codex prompts**** for:

- * re-extracting using committer date,
- * generating rate-normalized derived columns,
- * generating bot/no-bot ownership summaries,
- * and verifying BibTeX compilation.